

Item Inspection (Draft)

Connect Data Science

19 February, 2026

Table of Contents

1	II Dynamic Price and Sales Enhancement.....	6
1.1	1. Project Summary / Scope of the project	6
1.1.1	1.1. Problem Opportunity Statement	6
1.1.2	1.2. Project Scope.....	6
1.1.3	1.3. Outcome of Methods Work	6
1.2	2. Background revision	7
1.2.1	2.1. Overview and accents	7
1.2.2	2.2. Process flow	7
1.2.3	Calculate the values to be validated (Sales Rate and Price Rate)	14
1.3	3. Dynamic Sales Rates update.....	15
1.4	4. Dynamic Price Rates update with hold on same input price rates	15
1.5	5. Examples	16
2	II In Sirval - Method Description	18
2.1	Table of Contents.....	18
2.2	Rawdata Inspection	18
2.3	Item Inspection	18
2.4	The ITEM_INSPECTION process.....	19
2.5	Fuller: Inspection/Correction engine	19
2.6	Fuller: Reading Store information	20
2.7	Fuller: Item by Item checking	21
2.8	Fuller: Item by Item checking (sales rates)	21
2.9	Fuller: outlier check.....	22
2.10	Fuller: Exponential distribution.....	23
2.11	Fuller: outlier check.....	30
2.12	Fuller: outlier correction (sales rates).....	30
2.13	Fuller: outlier correction.....	31
2.14	Fuller: correction assessment.....	31
2.15	A hand calculated Example	34
2.16	The RAWDATA_CORRECTIONS table	37

2.17	Parameters for ITEM_INSPECTION	37
2.18	Item_Inspection and census	37
2.19	Conclusions.....	38
3	II LDI validation and correction add-on - prices.....	39
3.1	II LDI Validation and Correction Add-On – Prices	39
3.1.1	Project Summary / Scope of the Project.....	39
3.1.2	Preprocess	39
3.1.3	Validation and Correction Process	41
3.1.4	Output Data	41
4	II LDI validation and correction add-on - sales.....	43
4.1	Problem/ Opportunity	43
4.2	Project Scope	43
4.3	Proposal.....	43
4.3.1	Pre-process	43
4.3.2	Validation and correction during II.....	43
4.4	Sales calculations	43
4.4.1	Calculating differences for bounds	43
4.4.2	Calculating bounds	44
4.4.3	Calculating differences for bounds LDI items current period	44
4.4.4	Outlier detection.....	45
4.4.5	Corrections	45
4.5	New Items cross sectional validation.....	45
4.5.1	Detection and correction	45
4.6	Prices calculation.....	47
4.7	Parameters	47
4.8	Links.....	48
5	II SOP	49
5.1	Business Need	49
5.2	High-Level Process Flow	49
5.3	SOP Purpose	49
5.4	SOP Process Flow Chart.....	50
5.5	Detailed Steps	50

Item Inspection is a quality control process used to detect and correct unusual sales and price data at the item level in scanned retail data. It automatically reviews weekly store data to identify outliers – unusually high or low values that may be due to errors or inconsistencies.

Here's how it works:

- It checks for extreme values in **sales rates** and **prices**, comparing each item across multiple stores in a given period.
- The system uses statistical methods (like the *Fuller algorithm*) to flag and optionally correct these outliers.
- Recent improvements make this process more accurate by:
 - Repeating the correction loop to ensure consistency when multiple issues are found.
 - Applying the same correction to identical prices to avoid illogical differences in the output.
 - Dynamically updating input data after each correction to improve the next round of checks.

This ensures that the final data used for reporting or analysis is reliable, avoiding distortions that can result from raw errors or anomalies in the original retailer data.

1 II Dynamic Price and Sales Enhancement

1.1 1. Project Summary / Scope of the project

1.1.1 1.1. Problem Opportunity Statement

Current version of II relies on two consecutive rates in order to determine whether correction is needed or not. Inconsistent corrections are observed in cases when there is small difference between two observations after the first identified for correction. In the same time when it comes to prices, equal prices as input are corrected and output as different, which is not desired behavior for US team.

1.1.2 1.2. Project Scope

The request was to analyze the current Item Inspection methodology and to determine how to get an enhancement in the procedure which will ensure consistent detection and correction on all observations following the first identified as outlier.

NB! Out of scope:

- Currently number of observations for detection and correction is dependent on tail size only regardless of where in the set of observations the break appears

1.1.3 1.3. Outcome of Methods Work

As part of the request the following topics were reviewed:

- The statistical methodology and operational process of Item Inspection algorithm for High Sales Rates; both High and Low Price Rates
- Simulated extreme cases examples that replicate the issue at hand
- Root Cause of the inconsistency for detection, based on moving difference between rates differences

The following enhancement is suggested to address the identified root cause: iterative run of the process with dynamic update on the input values with the corrected ones, with constant correction for same input Price Rates.

1. Price Rates both left and right

- i. Same corrected rate if same initial price
- ii. No gaps in corrections (dynamic update of the tail)

2. Sales Rates

- i. No gaps in corrections (dynamic update of the tail)

1.2 2. Background revision

As mentioned before, **Item Inspection** is a quality control process used to detect and correct unusual sales and price data at the item level in scanned retail data. It automatically reviews weekly store data to identify outliers – unusually high or low values that may be due to errors or inconsistencies.

1.2.1 2.1. Overview and accents

1. Cross – sectional outlier detection on item level and correction with Fuller engine
2. Object of validation : Sample Scanning Data
3. Subjects of validation (outliers detection and correction):
 - i. Right tail of the distribution of the **sales rates**
 - ii. Left and Right tails of the distribution of the **prices**

1. Correction stage

- i. Fuller algorithm for correcting outliers (if needed)
- ii. Assess the impact of the expanded corrections for various IBDs
- iii. Corrections applied to data directly, not thru factor (insert into RAWDATA_CORRECTIONS)

1.2.2 2.2. Process flow

1.2.2.1 Define parameters

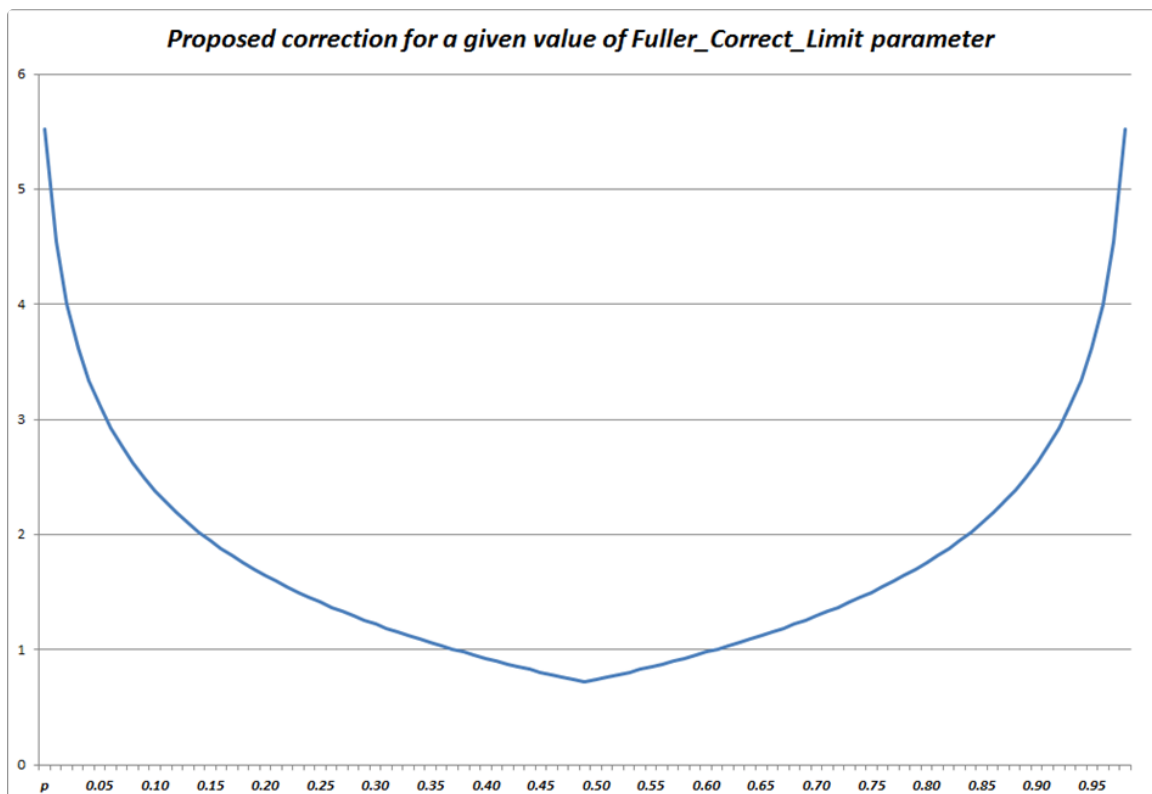
II parameters - prices

AC_PARAME TERTAG	De fau lts	Description
AVAIL_STOR ES_PCT	80	Minimum percentage of stores to be present/acceptable on the rawdata
CorrectRelse ExtraProm	1.5	Extra tolerance for promotions
CorrectRelse Tol_Price	1.5	z-value for the test on price
FilterCV_Pric e	0.1 5	Parameter used for shrinking the coefficient of variation to the prior (for checks on prices)

FilterTol_Price	2.5	Parameter used for calculating the critical ratio to the median (for checks on prices)
FullerCorrectLimit	0.9	Probability level for the F-test in the Fuller outlier correction procedure
FullerDetectLimit	0.995	Probability level for the F-test in the Fuller outlier detection procedure
FullerMaxTail	50	The maximum size to be considered as a tail
Min_rel_price_change	1.1	Minimal price correction (ratio between corrected and original data)

- AVAIL_STORES_PCT – expressed in %. This parameter is the minimum % of the acceptable stores in the current period that must be available out of:
 - Stores that were available in raw data in the previous period.
 - Stores that are available in raw data in the current period.
- Increasing this parameter value means we require a larger number of acceptable stores from either of the two sets of stores mentioned above for running item inspection, and vice versa. Suppose we have fast-changing samples from one period to another (e.g., new stores opening and/or stores closing). In that case, it is advisable to use a lower value for this parameter when running item inspection, for example, 60%.
- CorrectRelseToL_Price – this parameter is used during the assessment of the significance of proposed corrections on prices and the decision on whether to apply or not these corrections. Basically, the decision to apply the correction is based on a comparison between the correction's effect and a marginal value. If the impact exceeds the marginal value, the correction is applied. The larger this parameter's value is, the larger the marginal value is, and hence the greater the chance of ignoring the price correction, and vice versa.
- FilterCV_Price – this parameter is used during the process of checking whether to inspect the left/right tail of prices for outliers. The bigger the value of this parameter, the bigger the chance of skipping checking for outliers and vice versa.
- FilterTol_Price - this parameter is used during the process of checking whether to inspect the left/right tail of prices for outliers. The bigger the value of this parameter the bigger the chance to skip checking for outliers and vice versa (parameter has the same effect in item inspection as FilterCV_Price);

- **CorrectRelseExtraProm** – this parameter is used during the assessment of the significance of proposed corrections on both sales units and prices, and the decision on whether to apply or not these corrections. Its purpose is to increase further the marginal value against which the effect of the proposed correction is compared; i.e., in the case of promotional activities, it is natural to see larger sales, and we do not want to correct those, so we raise the limit. Suppose any promo activity is observed in the data (as part of item inspection). In that case, the weighted promo distribution is computed, and the proposed correction is compared to a marginal value, which is additionally multiplied by the value of this parameter. In the presence of promo activity, the bigger the value of this parameter, the bigger the chance to ignore the correction, and vice versa. In the absence of promo activity, this parameter is set to 1, and the marginal value, on which it is decided whether to apply a correction, is not further boosted.
- **FullerCorrectLimit** – this parameter has direct effect on the size of the correction proposed for each detected outlier. It is a probability and has the same effect on the proposed correction when set to p or $1-p$, where p is a number between 0 and 1. For example we would obtain the same result if item inspection were run with this parameter set to 0.1 and 0.9; 0.2 and 0.8, 0.3 and 0.7, 0.4 and 0.6. When the value of this parameter is a number above 0.5, the larger it is, the larger the proposed correction is, and vice versa. Respectively, when the value of this parameter is less than 0.5, the lower it is, the bigger the proposed correction is. The chart below illustrates how the proposed correction changes when changing the value of this parameter.



- **FullerDetectLimit**- this parameter has a direct effect on the decision if a data point (store/nan_key sales or price) is an outlier. It is a probability and has precisely the same effect

when set to p or $1-p$, where p is a number between 0 and 1. For example we would obtain the same result if item inspection were run with this parameter set to 0.1 and 0.9; 0.2 and 0.8, 0.3 and 0.7, 0.4 and 0.6. When the value of this parameter is a number above 0.5, then the bigger it is, the lower the chance of flagging a data point as an outlier, and vice versa. Respectively, when the value of this parameter is less than 0.5, the lower it is, the bigger the chance to flag a data point as an outlier. For example, if we are testing whether sales of 8 units are exceptional and they are not flagged as an outlier with FullerDetectLimit=0.8, but we do want this to be flagged as an outlier, then we should decrease the value of this parameter, but not below 0.5. Or, if we have a store with 25 sales units flagged as outliers with FullerDetectLimit=0.6 and corrected, but we do not want to correct this data point, then we should increase the value of this parameter.

- FullerMaxTail – this parameter determines the maximum number of data points within which to look for outliers. If we want to inspect more cases, this parameter must be set to a larger value; conversely, if we want to inspect fewer cases, it must be set to a smaller value.
- Min_rel_price_change - this parameter sets up a minimum correction rate for prices below which corrections are not applied, i.e., if the ratio between corrected and original sales value for a given store/nan_key is below this parameter, then the price correction is not applied. Since prices can be corrected on both the low and high sides, this parameter uses sales value to run the check, considering sales units, where price is computed as value/units.

1.2.2.2

II parameters - sales

AC_PARAMETERTAG	Defaults	Description
ACV_AddConstant	1000	Constant used in the denominator of the calculation of the sales rates
AVAIL_STORES_PCT	80	Minimum percentage of stores to be present/acceptable on the rawdata
CorrectRelseExtraProm	1.5	Extra tolerance for promotions
CorrectRelseToL_Val	1.5	z-value for the test on value
FilterCV_Vol	0.35	Parameter used for shrinking the coefficient of variation to the prior (for checks on volumes)

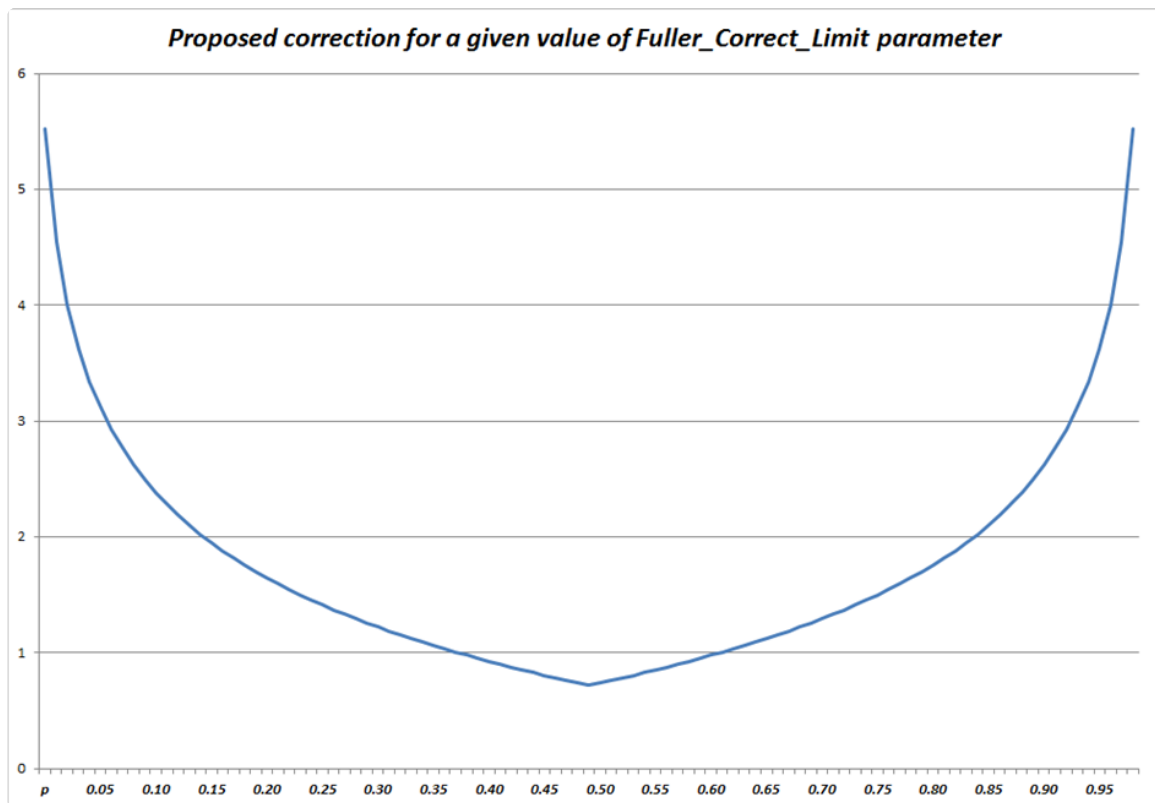
FilterToI_Vol	2.5	Parameter used for calculating the critical ratio to the median (for checks on volumes)
FullerCorrectLimit	0.9	Probability level for the F-test in the Fuller outlier correction procedure
FullerDetectLimit	0.995	Probability level for the F-test in the Fuller outlier detection procedure
FullerMaxTail	50	The maximum size to be considered as a tail
Min_max_volume	10	Minimal sales to run the inspection for
Min_rel_vol_change	1.1	Minimal sales correction (ratio between corrected and original data)

- ACV_AddConstant – this parameter has no direct effect on the inspection process and proposed corrections. It is used during the transformation of sales units of each store/nan_key before running the inspection.
- AVAIL_STORES_PCT – expressed in %. This parameter is the minimum % of the acceptable stores in the current period that must be available out of:
 - Stores that were available in raw data in the previous period;
 - Stores that are available in raw data in the current period;

Increasing this parameter value means we require a larger number of acceptable stores from either of the two sets of stores mentioned above for running item inspection, and vice versa. I.e., suppose we have fast-changing samples from one period to another (e.g., new stores opening and/or stores closing). In that case, it is advisable to use a lower value for this parameter when running item inspection, for example, 60%.

- CorrectRelseExtraProm – this parameter is used during the assessment of the significance of proposed corrections on both sales units and prices, and the decision on whether to apply or not these corrections. Its purpose is to increase further the marginal value against which the effect of the proposed correction is compared; i.e., in the case of promotional activities, it is natural to have higher sales, and we do not want to correct those, so we raise the limit. Suppose any promo activity is observed in the data (as part of item inspection). In that case, the weighted promo distribution is computed, and the proposed correction is compared to a marginal value, which is additionally multiplied by the value of this parameter. In the presence of promo activity, the bigger the value of this parameter, the bigger the chance to ignore the correction, and vice versa. In the absence of promo activity, this parameter is set to 1, and the marginal value, on which it is decided whether to apply a correction, is not further boosted.

- **CorrectRelseTol_Val** - this parameter is used during the assessment of the significance of proposed corrections on sales and the decision on whether to apply or not these corrections. Basically, the decision to apply the correction is based on a comparison between the correction's effect and a marginal value. If the impact exceeds the marginal value, the correction is applied. The larger this parameter's value is, the larger the marginal value is, and hence the greater the chance of ignoring the sales correction, and vice versa.
- **FilterCV_Vol** - this parameter is used during the process of checking whether to inspect the right tail of sales for outliers. The bigger the value of this parameter, the bigger the chance of skipping checking for outliers and vice versa.
- **FilterTol_Vol** - this parameter is used during the process of checking whether to inspect the right tail of sales for outliers. The bigger the value of this parameter, the bigger the chance to skip checking for outliers and vice versa (parameter has the same effect in item inspection as **FilterCV_Vol**).
- **FullerCorrectLimit** – this parameter has a direct effect on the size of the correction proposed for each detected outlier. It is a probability and has precisely the same impact on the suggested correction when set to either p or $1-p$, where p is a number between 0 and 1. For example, we would obtain the same result if item inspection is run with this parameter set to 0.1 and 0.9; 0.2 and 0.8, 0.3 and 0.7, 0.4 and 0.6. When the value of this parameter is a number above 0.5, then the bigger it is, the bigger the proposed correction is, and vice versa. Respectively, when the value of this parameter is less than 0.5, the lower it is, the bigger the proposed correction is. The chart below illustrates how the proposed correction changes as the value of this parameter is varied.



- FullerDetectLimit- this parameter has a direct effect on the decision if a data point (store/ nan_key sales or price) is an outlier. It is a probability and has precisely the same effect when set to p or $1-p$, where p is a number between 0 and 1. For example, we would obtain the same result if item inspection is run with this parameter set to 0.1 and 0.9; 0.2 and 0.8, 0.3 and 0.7, 0.4 and 0.6. When the value of this parameter is a number above 0.5, then the bigger it is, the lower the chance of flagging a data point as an outlier, and vice versa. Respectively, when the value of this parameter is less than 0.5, the lower it is, the greater the chance of flagging a data point as an outlier. For example, if we are testing whether sales of 8 units are exceptional and they are not flagged as an outlier with FullerDetectLimit=0.8, but we do want this to be flagged as an outlier, then we should decrease the value of this parameter, but not below 0.5. Or, if we have a store with 25 sales units flagged as outliers with FullerDetectLimit=0.6 and corrected, but we do not want to correct this data point, then we should increase the value of this parameter.
- FullerMaxTail – this parameter determines the maximum number of data points within which to look for outliers. If we want to inspect more cases, this parameter has to be set to a larger value, and vice versa.

- **Min_max_volume** – this parameter determines the minimal threshold of sales below which item inspection will not run, i.e., item inspection will skip any nan_key, for which the maximum sales across all stores are lower than this parameter. If we want to inspect only the nan_keys with high sales, we should set this value to a high number; otherwise, we should set it to a low number.
- **Min_rel_vol_change** – this parameter sets up a minimum correction rate for sales below which corrections are not applied, i.e., if the ratio between corrected and original sales for a given store/nan_key is below this parameter, then the sales correction is not applied.

1.2.3 Calculate the values to be validated (Sales Rate and Price Rate)

i.

$$\text{Sales Rate} = \frac{ACV_AddConstant * NC_SALES}{(ACV_AddConstant + NC_ACV)}$$

ii.

$$\text{Price Rate} = \frac{(NC_SALES * NC_PRICE + 0.001)}{(NC_SALES + 0.001)}$$

Calculate min, max, median, 1st and 3rd quartile, OCV, SCV

$$OCV = \frac{(Q3 - Q1)}{(Q2 * 1.34898)}$$

Where:

OCV - Observed CV based on IQR

Q1 – 1st quartile

Q3 – 3rd quartile

Q2 – median

$$SCV = \frac{(OCV * n + FilterCV_Price * 30)}{(n + 30)}$$

Where:

FilterCV_Price/FilterCV_Vol – is predefined parameter. This parameter is used during the process of checking whether to inspect the left/right tail of prices (right tail of Sales rates resp.) for outliers. The bigger the value of this parameter the bigger the chance to skip checking for outliers and vice versa

n – Number of observations

Define tail length to inspect – up to predefined parameter(currently 50)

Define number of observations from the tail to test for correction – if less than 6 obs in tail – 1; if less than 15 – 2; else 1/10 of number of observations in tail(floored)

Calculations (refer to attachment)

- i. estimates needed for the test statistics – b, b1, b2
- ii. Calculate the Fval observed
- iii. Calculate Fcrit
- iv. Calculate Fcorrect
- v. Calculate expected gap
- vi. Apply correction

1.3 3. Dynamic Sales Rates update

The enhancement is affecting the first stage of II process (Fuller: Inspection/Correction engine) by adding iterative element for the engine run. The process is run as many times as many observations where calculated to be inspected. If correction is proposed for the first observation from the list, in the second run the corrected value will be placed as input value. And so on and so forth.

1.4 4. Dynamic Price Rates update with hold on same input price rates

For the price rates the enhancement is taking same logic as for the sales rates but at the same time it is accounting for equal input prices. In case of two or more consecutive observations with the same input price value – the same correction is applied to the following one or more observations, thus

keeping the output prices equal. Fuller Detection Engine is run iteratively as many times as number of observations to check in tail.

1.5 5. Examples

Price Rates – current process tail to inspect/correct

AC_NSHOPID	F_NAN_KEY	NC_SALES	NC_PRICE	C_PRICE	AC_IBD
280003N916	12886395	1	37	21.06	SFLG:3
280003N916	12886395	1	37	37	SFLG:3
280001NJ27	12886395	1	38	38	SFLG:3
280001NJ27	12886395	1	98	42.68	SFLG:3
980004CY56	12886395	1	256	113.05	SFLG:3

Price Rates dynamic update – result tail inspect/correct

NC_PERIODID	AC_NSHOPID	F_NAN_KEY	NC_SALES	NC_PRICE	C_PRICE
1720	280003N916	12886395	1	37	21.06
1720	280003N916	12886395	1	37	21.06
1720	280001NJ27	12886395	1	38	22.83
1720	280001NJ27	12886395	1	98	25.56
1720	980004CY56	12886395	1	256	31.18

For step by step please refer to step by step in attached document.

Tech: R

Scripts : [link](#)¹

1. <https://drive.google.com/open?id=1QrnvDk-52yRuctjcdGE6tEaNYqQpkKhc>

2 II In Sirval - Method Description

2.1 Table of Contents

- Raw data Inspection
- Item Inspection
- Fuller-engine
- Raw data Corrections table
- Parameters for Item Inspection
- Conclusion

2.2 Rawdata Inspection

Sirval **has two processes to inspect the Rawdata at the shop level:**

1. FOVA: Formal Validation of Volumetric Data
2. CFOVA: Formal Validation of Causal data

Sirval **has three processes to inspect the rawdata below the shop level:**

1. PSG: Cross-sectional check of Price variation at the item-level
2. ITEM_INSPECTION:
 - a. Cross-sectional outlier detection at the item level
 - b. Correction
3. MODULE_INSPECTION: (NIY)
 - a. Cross-sectional outlier detection at the module level
 - b. No Correction

2.3 Item Inspection

- Cross-sectional
- Outlier detection and correction with Fuller
- Impact assessment on EBDs
- two possible sources of inspection:
 - Rawdata: Inspect all items on the Rawdata:
 - i. 1 period
 - ii. Store Filter

iii. Module Filter

- Inquiry table: Cfr. above, but only inspect those items that were put on the Inquiry table (NIY)

2.4 The ITEM_INSPECTION process

Process Flow:

- Fetch parameter values
- Resolve Store filter
- Pick up filtered acceptable stores
- Some checks on %acceptable stores compared to:
 - Received stores
 - Previous period
- Resolve module filter
- Pick up Volumetric Rawdata (period, selected stores, selected modules)
- Set the Causal flag for each item (period shop) for which there is causal activity for that Nan (period, shop)
- Launch the FULLER inspection/correction engine by passing:
 - Parameters
 - Selected Stores
 - Selected Rawdata

2.5 Fuller: Inspection/Correction engine

- **Used by 2 processes**
 - ITEM_INSPECTION
 - MODULE_INSPECTION
- **Programmed in JAVA (not PL/SQL)**
- **The FULLER engine:**
 - Pre-processing
 - Uses Fuller's algorithm for finding outliers
 - Uses Fuller's algorithm for correcting outliers (if that is needed, cfr. Item_inspection)
 - assesses the impact of the (expanded) corrections for various EBD's.
 - Inserts "significant" corrections in the RAWDATA_CORRECTIONS table
- **Process flow:**

Reading store information (= loop over stores)

- Build Cells
- Build IBD's

Item by Item checking (=loop over items):

- Check:
 - Check Right-tail of the distribution of the sales rates ($=1000 \times \text{Units}/(\text{ACV} + \text{ACV_add_constant})$)
 - Check Right-tail of the distribution of the prices
 - Check the Left-tail of the distribution of the prices
- Propose corrections
- Assess impact of corrections
- Insert into RAWDATA_CORRECTIONS

2.6 Fuller: Reading Store information

Store:1200003485	Acv:300.0	Cell:1	Ibd:[1, 2, 3, 0]	Xf: 60.03	CellSACv: 300.00	CellUAcv: 18009.50
Store:1200003483	Acv:243.0	Cell:1	Ibd:[4, 2, 3, 0]	Xf: 60.03	CellSACv: 543.00	CellUAcv: 32597.20
Store:1200003550	Acv:288.0	Cell:1	Ibd:[4, 2, 3, 0]	Xf: 60.03	CellSACv: 831.00	CellUAcv: 49886.33
Store:1200003553	Acv:252.0	Cell:1	Ibd:[5, 2, 3, 0]	Xf: 60.03	CellSACv: 1083.00	CellUAcv: 65014.31
Store:1200003559	Acv:283.0	Cell:1	Ibd:[4, 2, 3, 0]	Xf: 60.03	CellSACv: 1366.00	CellUAcv: 82003.27
Store:1200003786	Acv:338.0	Cell:1	Ibd:[5, 2, 3, 0]	Xf: 60.03	CellSACv: 1704.00	CellUAcv: 102293.98
Store:1200003788	Acv:269.0	Cell:1	Ibd:[1, 2, 3, 0]	Xf: 60.03	CellSACv: 1973.00	CellUAcv: 118442.50
Store:1200003561	Acv:363.0	Cell:1	Ibd:[1, 2, 3, 0]	Xf: 60.03	CellSACv: 2336.00	CellUAcv: 140234.00
Store:1200003993	Acv:288.0	Cell:2	Ibd:[4, 2, 3, 0]	Xf: 86.46	CellSACv: 288.00	CellUAcv: 24901.72

Note 1: CellsAcv and Cell Universe ACV are cumulative per cell

Note 2: IBD's defined by a parameter listing stores fields

Cells :2				
Cell	Cell Label	Count	Universe Acv	X-factor
1	12034	8	140234.00	60.03
2	12033	1	24901.72	86.46
IBD :5				
IBD	IBD Label	Count		
1	IBD1_NL3	3		
2	IBD2_GROCERY_TRAD	9		
3	IBD3_NL	9		
4	IBD1_NL4	4		
5	IBD1_NL5	2		

Note: Cell X-factors are recalculated as $\text{CellUAcv}/\text{CellSAcv}$

2.7 Fuller: Item by Item checking

<<GROUP>> nan=8119001911 cref=5410013262008 crefsuf= period=1097								
<<GROUP>> nan=8119002263 cref=8710395022106 crefsuf= period=1097								
<<GROUP>> nan=8119002265 cref=8710395021109 crefsuf= period=1097								
VOLUME:	0.75	1.55	1.55	2.31	2.41	3.12	3.15	3.99
q0=	0.75							
q1=	1.55							
q2=	2.36							
q3=	3.13							
q4=	3.99							
MAXIMUM / MEDIAN=	1.692	CRIT_RATIO=	1.952	(0.497)				
PRICE:	1.67	1.76	1.76	1.79	1.79	1.79	1.81	1.90
q0=	1.67							
q1=	1.76							
q2=	1.79							
q3=	1.80							
q4=	1.90							
MAXIMUM / MEDIAN=	1.061	CRIT_RATIO=	1.304	(0.015)				
MEDIAN / MINIMUM=	1.076	CRIT_RATIO=	1.304	(0.015)				

Note: under VOLUMES are the sales rates

$$=1000 \times \text{Units} / (\text{ACV} + \text{ACV_ADDconstant})$$

2.8 Fuller: Item by Item checking (sales rates)

- If too few observations: skip
- If no variation: skip
- Compute:
 - Q0: minimum

- Q1: first quartile
- Q2: median
- Q3: third quartile
- Q4: maximum
- **Estimate coefficient of variation via interquartile range and median:**
 - $OCV = (Q3 - Q1) / (1.34898 \times Q2)$
- **Shrink to prior**
- **Compute critical ratio to the median**

If maximum > Median x Critical_Ratio_to_the_median then check for outliers on the right tail (=about 1/3 of the total distribution), otherwise skip.

- Note 1: For prices both sides of the distributions are tested. The test on the left tail is by evaluating Median / Minimum
- Note 2: If outlier check is needed some parameters of the Weibull distribution for the observations in the tail are printed (regression of the order statistics). This can be used to assess whether the assumption of an exponential distribution holds
- Note 3: The number of outliers to check for is determined

2.9 Fuller: outlier check

- **If tail it too small: skip**
- **If tail has no variation: skip**
- **LOOP over the number of outliers that we need to check for:**
 - Assume an exponential distribution (in the tail)
 - Consider the gaps between points
 - Compute test statistic
 - Evaluate test statistic to a F-distribution

2.10 Fuller: Exponential distribution

Density function:

$$f(x) = P[X = x] = \frac{1}{b} e^{-x/b} \quad \text{for } x \geq 0$$

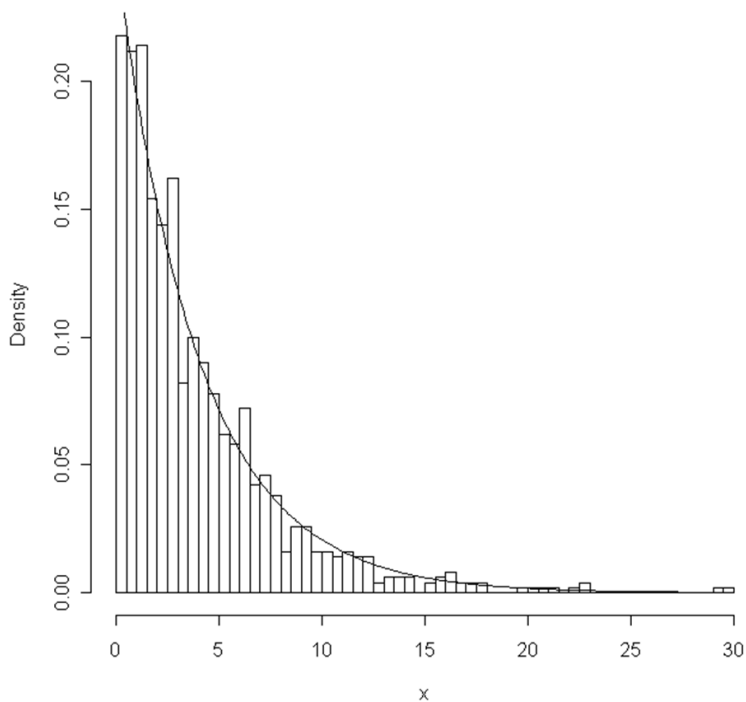
Cumulative distribution function:

$$F(x) = P[X \leq x] = 1 - e^{-x/b} \quad \text{for } x \geq 0$$

Common Statistics:

$$\begin{cases} E(x) = b \\ Var(x) = b^2 \\ CV(x) = 1 \end{cases}$$

Density function from an exponential distribution with $b=4$ and a histogram of a sample ($n=1000$) from such a distribution



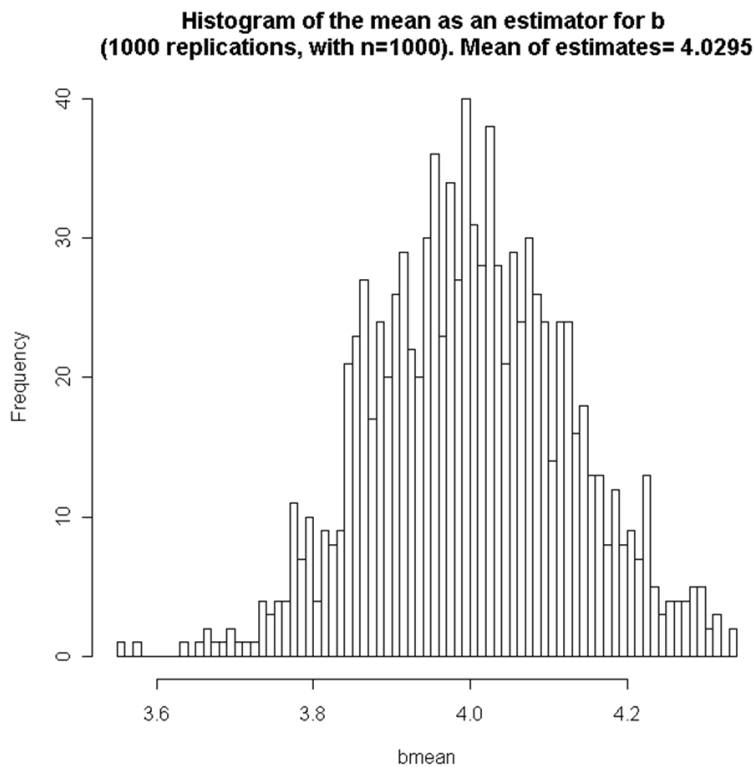
If we have a sample of size n , how can we estimate b ?

Just take the average:

$$\hat{b} = \frac{1}{n} \sum_{i=1}^n X_{(i)}$$

In this case: 3.861186

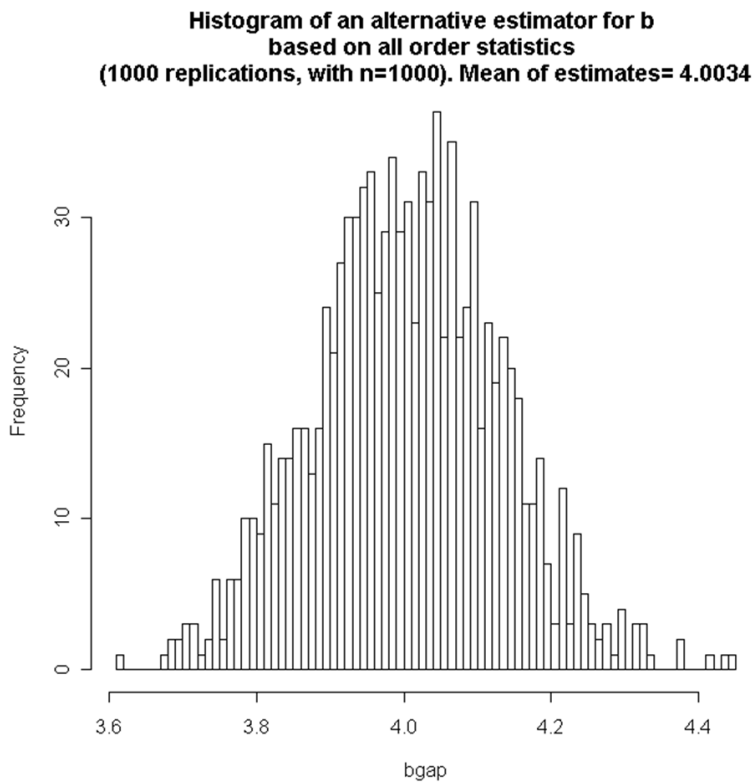
To demonstrate that this works in general, this has been replicated 1000 times, yielding the following distribution



There are alternative estimators for b . Consider for instance the gaps between the order statistics:

$$\hat{b} = \frac{1}{n-1} \sum_{i=2}^n (n-i+1)(X_{(i)} - X_{(i-1)})$$

The i -th order statistic here simply means the i -th element after all elements were ordered from the low to high. Doing the same exercise gives:



An estimator for b, based on the first n-1 order statistics (slightly shrunk to the average to avoid division by zero) is:

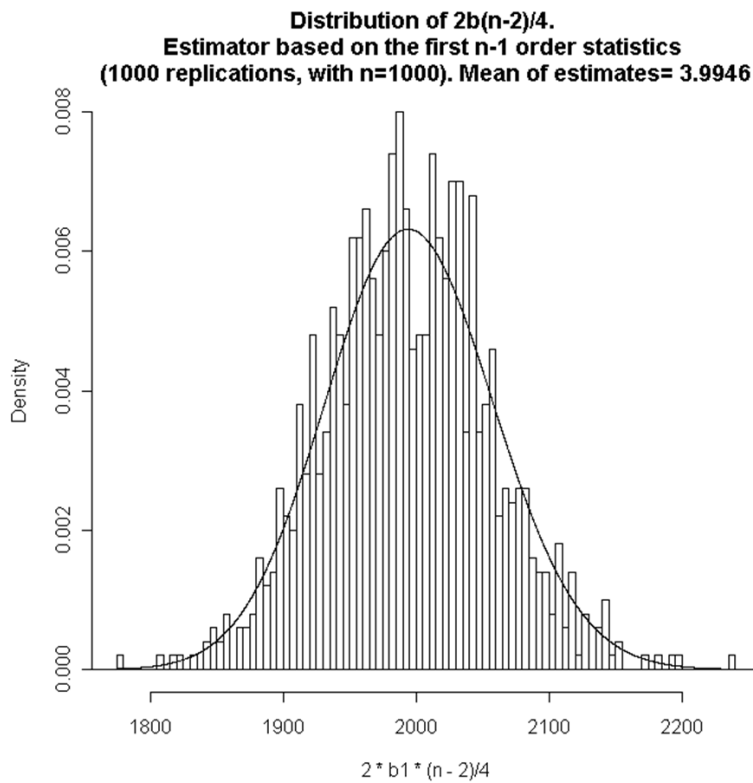
$$\hat{b}_1 = 0.999 \times \frac{1}{n-2} \sum_{i=2}^{n-1} (n-i+1)(X_{(i)} - X_{(i-1)}) + 0.001 \times \hat{b}$$

Disregarding the shrinkage here, we can do the same exercise as above. This time we don't plot b1 as

$$\frac{2(n-2)\hat{b}_1}{b}$$

such, but:
degrees of freedom:

which can be shown to follow a Chi-square distribution with 2(n-2)



Somewhat surprisingly, simply taking the gap between the last two order statistics gives an estimator for b :

$$\hat{b}_2 = X_{(n)} - X_{(n-1)}$$

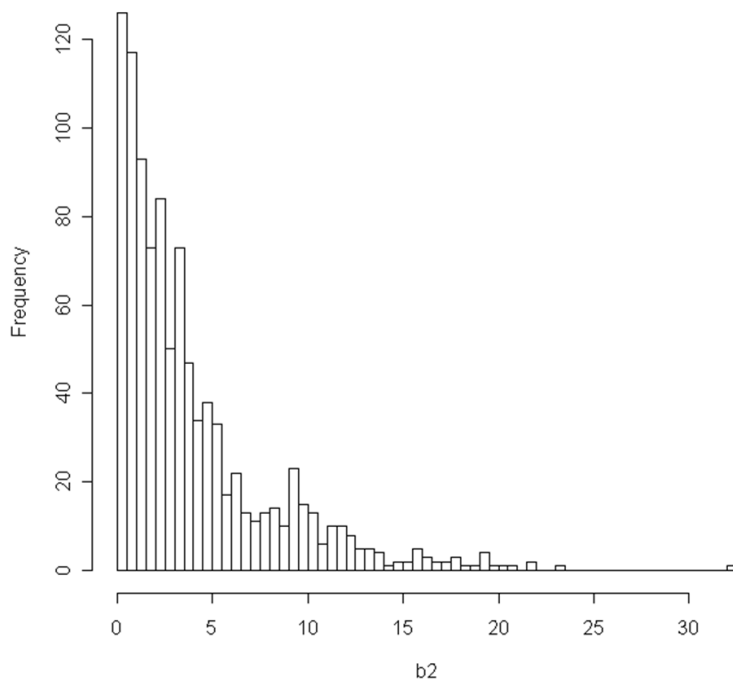
In *surval* this estimator is shrunk to the average (to avoid division by zero) :

$$\hat{b}_2 = 0.999(X_{(n)} - X_{(n-1)}) + 0.001 \times \hat{b}$$

Disregarding the shrinkage we can repeat the exercise

Note that the values of this estimator varies more, but the mean of the estimates seems to be OK.

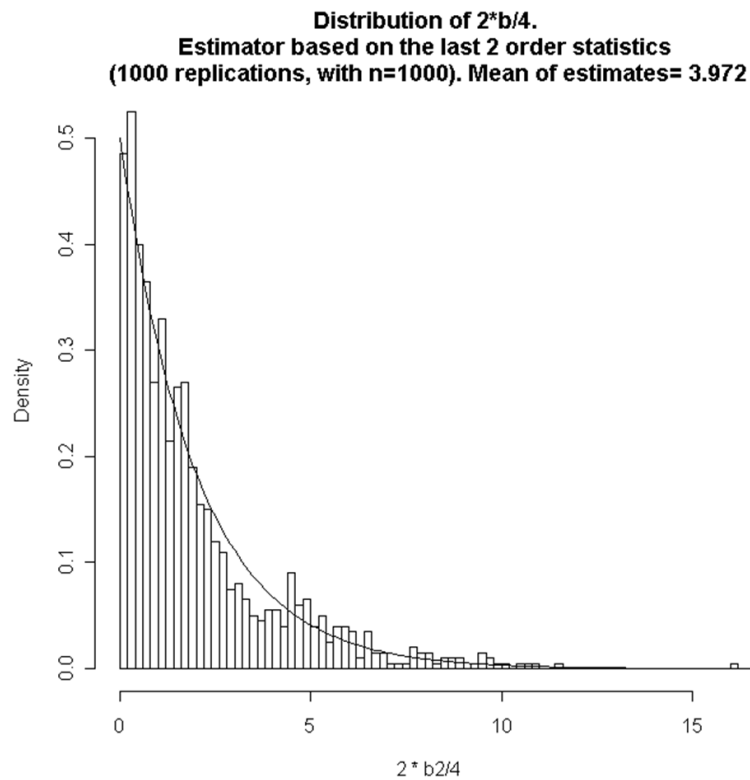
**Histogram of the "last gap" as an estimator for b
(1000 replications, with n=1000). Mean of estimates= 3.972**



$$\frac{2\hat{b}_2}{b}$$

It can be shown that:

follows a Chi-square distribution with 2 degrees of freedom:

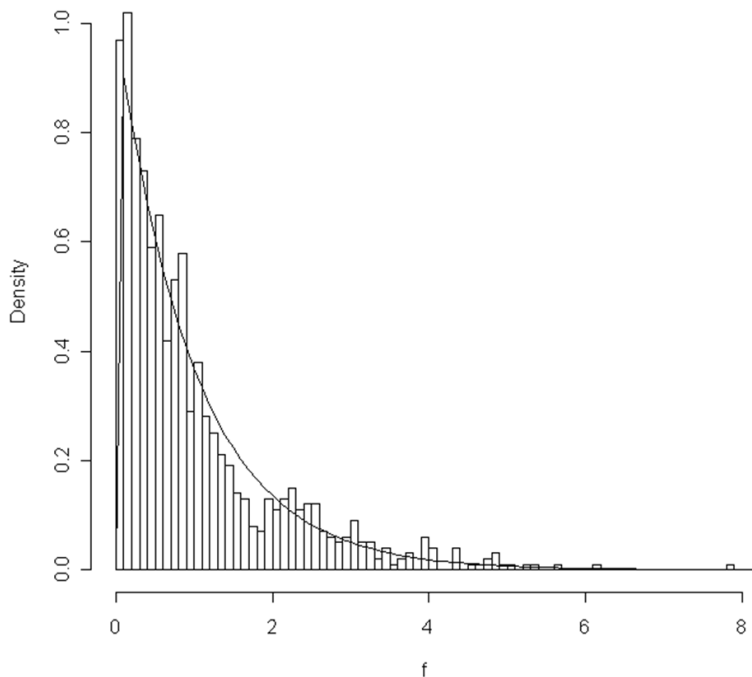


The ratio of b_2 and b_1 can be shown to be F-distributed with respectively 2 and $2(n-2)$ degrees of freedom.

This allows us to specify a critical value for the test statistic in order to test whether the extreme tail of the distribution is in line with the expectations according to an exponential distribution.

In area 1 we currently use a probability level of 0.995.

Notice also that in Sirval we use $2(n-2) - 0.5$ degrees of freedom for the denominator to become more tolerant for very small n .

Distribution of b_2/b_1 

If the test-statistic is more than the critical value a correction is proposed based on an expected gap as:

$$X_{(n)} = X_{(n-1)} + \hat{b}_1 \times \text{Crit_F}$$

in which Crit_F is a critical value on the F-distribution with 2 and $2(n-2)$ degrees of freedom.

Note that the probability level for the critical value for a correction does not have to be the same as the one used for detection.

- **QUESTION:** That's all very fine as long as the distribution I'm interested in is exponentially distributed and this is seldom the case in our business. Is the usefulness of Fuller not very limited from this perspective?
- **ANSWER:** The TAIL of the distribution you're interested in can often be approximated by an exponential distribution (cfr. Ludo's document).
- **Note:** The previous explanation can easily be generalized to more than 1 outlier
- **Note:** See hand calculated example later

2.11 Fuller: outlier check

<<GROUP>> nan=8123003428 cref=8711000003145 crefsuf= period=1097									
VOLUME:	2.33	2.33	7.24	11.18	15.70	15.76	19.49	21.54	36.68
q0=	2.33								
q1=	7.24								
q2=	15.70								
q3=	19.49								
q4=	36.68								
MAXIMUM / MEDIAN= 2.337 CRIT_RATIO= 2.007 (0.578)									
check tail : RIGHT nc=5 : T=1									
CHECKED_TAIL :	15.70	15.76	19.49	21.54	36.68				

Note: No outlier according to Fuller. Here this fact suppresses the relevant output (SW was adapted).

2.12 Fuller: outlier correction (sales rates)

- If previous test signals « significance » a corrected value (in this case a corrected sales-rate) is proposed using the value we would expect (winsorizing the gap)
- If there are proposed corrections then:
 - translate the proposed corrected sales rates back to units ($\text{New Sales_rate} \times (\text{ACV} + \text{acv_addconstant}) / 1000$)
 - Test whether current units are integers. If yes then randomround
 - Recalculate the value by imposing the old price
 - The proposed correction will then be evaluated by assessing its impact on projected figures (cfr. Later)

2.13 Fuller: outlier correction

<<GROUP>> nan=8118001974 cref=8711000008140 crefsuf= period=1097				
VOLUME:	0.78	0.78	0.80	4.40
q0=	0.78			
q1=	0.78			
q2=	0.79			
q3=	2.60			
q4=	4.40			
MAXIMUM / MEDIAN=	5.569	CRIT_RATIO=	2.276	(1.713)
check tail : RIGHT	nc=4	:	T=1	
CHECKED_TAIL :	0.78	0.78	0.80	4.40
FULLER_CORRECT	t=1			
b1=	0.029	b2=	3.595	
fval=	122.7124	qfcrit=	47.6730	qfcorr= 4.3371
CORRECTED_TAIL:	0.78	0.78	0.80	0.93

2.14 Fuller: correction assessment

- If a correction was proposed its impact is assessed by looking at the effect on expanded figures for the different participating IBD's.
- LOOP over IBD's and compute for each IBD:
 - winsorized projected total value + relative standard error
 - winsorized projected total number of units + relse
 - projected price (projected total value / projected total number of units) + relse
 - projected ACV weighted selling distribution
 - projected ACV weighted promo distribution
 - number of stores selling
 - total number of stores in sample
- Check whether the corrections above are « significant » by looping over all stores with a correction and for each of these stores we loop over the relevant IBD's. If they are significant for 1 or more IBD's then it will be applied:
 - $ValDiff = xf * (original_val - corr_val) / proj_total_val_IBD$
 - $VolDiff = xf * (original_vol - corr_vol) / proj_total_vol_IBD$
 - $PDiff = (1 + ValDiff) / (1 + VolDiff) - 1$
 - Compute extra tolerance if item has promotional activity
 - $TolVal = z_value_val \times relse_val_lbd \times Extra_Tol$
 - $TolPrice = z_value_price \times relse_price_lbd \times Extra_Tol$
 - If $abs(ValDiff) > TolVal$ then SIGNIFICANT
 - If $abs(Pdiff) > TolPrice$ then SIGNIFICANT

There are corrections :								

<<GROUP>> nan=8124005030 cref=5410013175001 crefsuf= period=1097								
+-----+								
C	Store	Cell Label	X-factor	ACV	Units	Value	Price	
+-----+								
	1200003483	12034	60.03	243.00	1.	1.45	1.45	
	1200003550	12034	60.03	288.00	1.	1.59	1.59	
	1200003788	12034	60.03	269.00	1.	1.57	1.57	
	1200003993	12033	86.46	288.00	1.	1.59	1.59	
	1200003553	12034	60.03	252.00	1.	1.61	1.61	
1	1200003559	12034	60.03	283.00	2.	3.18	1.59	
+-----+								

Note: If Verbose=0 the Reports only contain assessed items

<<GROUP>> nan=8124005030 cref=5410013175001 crefsuf= period=1097						
VOLUME:	0.78	0.78	0.79	0.80	0.80	1.56
q0=	0.78					
q1=	0.78					
q2=	0.79					
q3=	0.80					
q4=	1.56					
MAXIMUM / MEDIAN=	1.965	CRIT_RATIO=	1.740	(0.026)		
check tail : RIGHT nc=5 :	T=1					
CHECKED_TAIL :	0.78	0.79	0.80	0.80	1.56	
WEIBULL_PARMS : a=	0.240	b=	0.464	r2=	0.693	
FULLER_CORRECT t=1						
b1=	0.030	b2=	0.754			
fval=	24.9306	qfcrit=	17.2566	qfcorr=	3.4508	
CORRECTED_TAIL:	0.78	0.79	0.80	0.80	0.91	
PRICE:	1.45	1.56	1.59	1.59	1.59	1.61
q0=	1.45					
q1=	1.56					
q2=	1.59					
q3=	1.59					
q4=	1.61					
MAXIMUM / MEDIAN=	1.014	CRIT_RATIO=	1.317	(0.011)		
MEDIAN / MINIMUM=	1.094	CRIT_RATIO=	1.317	(0.011)		

..Corrected store :1200003559 IBD=[4, 2, 3, 0]			
-->>Affected IBD-s : [4, 3, 2, 0]			
-->> IBDStats : 4 : IBD1_NL4			
-->>	Distr :	1.0000	(4.0/4.0)
-->>	Promo :	0.0000	
-->>	units :	266.5593	relse : 0.2717
-->>	value :	415.1822	relse : 0.2690
-->>	price :	1.5576	relse : 0.0175
-->> IBDStats : 3 : IBD3_NL			
-->>	Distr :	0.6361	(6.0/9.0)
-->>	Promo :	0.0000	
-->>	units :	386.6227	relse : 0.2604
-->>	value :	605.8669	relse : 0.2593
-->>	price :	1.5671	relse : 0.0134

-->> IBDStats : 2 : IBD2_GROCERY_TRAD			
-->>	Distr :	0.6361	(6.0/9.0)
-->>	Promo :	0.0000	
-->>	units :	386.6227	relse : 0.2604
-->>	value :	605.8669	relse : 0.2593
-->>	price :	1.5671	relse : 0.0134
-->> IBDStats : 0 : ***TOTAL***			
-->>	Distr :	0.6361	(6.0/9.0)
-->>	Promo :	0.0000	
-->>	units :	386.6227	relse : 0.2604
-->>	value :	605.8669	relse : 0.2593
-->>	price :	1.5671	relse : 0.0134

2.15 A hand calculated Example

There are corrections :								

<<GROUP>> nan=8123003430 cref=8711000003244 crefsuf= period=1097								
+-----+								
C	Store	Cell Label	X-factor	ACV	Units	Value	Price	
+-----+								
	1200003559	12034	60.03	283.00	5.	17.90	3.58	
	1200003485	12034	60.03	300.00	6.	20.67	3.44	
	1200003553	12034	60.03	252.00	11.	39.88	3.63	
	1200003788	12034	60.03	269.00	11.	37.89	3.44	
	1200003483	12034	60.03	243.00	15.	48.74	3.25	
	1200003550	12034	60.03	288.00	17.	67.85	3.99	
	1200003993	12033	86.46	288.00	17.	67.85	3.99	
	1200003786	12034	60.03	338.00	18.	62.00	3.44	
1	1200003561	12034	60.03	363.00	100.	326.27	3.26	
+-----+								

<<GROUP>> nan=8123003430 cref=8711000003244 crefsuf= period=1097									
VOLUME:	3.90	4.62	8.67	8.79	12.07	13.20	13.20	13.45	73.37
q0=	3.90								
q1=	8.67								
q2=	12.07								
q3=	13.20								
q4=	73.37								
MAXIMUM / MEDIAN= 6.080 CRIT_RATIO= 1.834 (0.278)									
check tail : RIGHT nc=5 : T=1									
CHECKED_TAIL :	12.07	13.20	13.20	13.45	73.37				
WEIBULL_PARMS : a=20.313 b= 0.399 r2= 0.553									
FULLER_CORRECT t=1									
b1=	1.692	b2=	59.871						
fval=	35.3798	qfcrit=	17.2566	qfcorr=	3.4508				
CORRECTED_TAIL:	12.07	13.20	13.20	13.45	19.29				

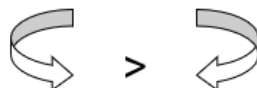
Note: $(100 \times 1000)/(363 + 1000) = 73.37$

Note: $73.37/12.07=6.080 > 1.834$: Let Fuller check for outlier(s)

i	n-i+1	X	Gap	Gap*(n-i+1)	Sum	Average	Shrunked
1	5	12.07					
2	4	13.2	1.13	4.52			
3	3	13.2	0	0			
4	2	13.45	0.25	0.5	5.02	1.6733	1.6879
5	1	73.37	59.92	59.92	59.92	59.92	59.8763
					64.94	16.235	

b1 points to 1.6879, b2 points to 59.8763, and b points to 16.235.

	Crit_F (2,6) df	Crit_F Correct	Expected gap	Proposed correction
Fval	35.47396	14.54418	3.463299	5.845685033
				19.29569


 $13.45 + (3.46 \times 1.6879)$

PRICE:	3.25	3.26	3.44	3.44	3.44	3.58	3.63	3.99	3.99
q0=	3.25								
q1=	3.44								
q2=	3.44								
q3=	3.63								
q4=	3.99								
MAXIMUM / MEDIAN=	1.159	CRIT_RATIO=	1.311	(0.039)					
MEDIAN / MINIMUM=	1.060	CRIT_RATIO=	1.311	(0.039)					

Note: No price problems

-->> IBDStats : 1 : IBD1_NL3
-->> Distr : 1.0000 (3.0/3.0)
-->> Promo : 0.0000
-->> units : 2641.3939 relse : 0.5103
-->> value : 8803.2653 relse : 0.4998
-->> price : 3.3328 relse : 0.0171
-->> IBDStats : 0 : ***TOTAL***
-->> Distr : 1.0000 (9.0/9.0)
-->> Promo : 0.0000
-->> units : 8073.3781 relse : 0.1295
-->> value : 28859.8615 relse : 0.1223
-->> price : 3.5747 relse : 0.0260
store=1200003561

Note: 2 other participating IBDs are the same as TOTAL

1 :IBD1_NL3	:	VALUE (1.624 / 0.750)	PRICE (-0.013 / 0.026)
2 :IBD2_GROCERY_TRAD	:	VALUE (0.495 / 0.183)	PRICE (-0.031 / 0.039)
3 :IBD3_NL	:	VALUE (0.495 / 0.183)	PRICE (-0.031 / 0.039)
0 :***TOTAL***	:	VALUE (0.495 / 0.183)	PRICE (-0.031 / 0.039)
>>>> store=1200003561 nan=8123003430 cref=8711000003244 crefsuf= period=1097			
CORRECT UNITS/CONV= 100.0000 => 27.0000			
CORRECT VALUE= 326.2680 => 88.0924			
CORRECT PRICE= 3.2627 => 3.2627			

Note 1: $0.495 = (326.27 - 88.09) \times 60.03 / 28859.86$

Note 2: $0.183 = 1.5 \times 0.1223 \times 1$

Conclusion: $0.495 > 0.183 \Rightarrow$ Effect for Total IBD

2.16 The RAWDATA_CORRECTIONS table

OP	TICKET	AC_PROCESSID	PER	NSHOPID	AC_CREF	NC_NAN	Conv	Slot1	Slot2	Slot4
M	-1109545	RAWDATA_INSPECTION	1097	1200003561	8711000008140	8118001974	1	2	1.806	0.903
M	-1109545	RAWDATA_INSPECTION	1097	1200003561	8711000003244	8123003430	1	27	88.0924	3.2627
M	-1109545	RAWDATA_INSPECTION	1097	1200003788	87133127	8248001737	1	1	1.3114	1.3114
M	-1109545	RAWDATA_INSPECTION	1097	1200003786	8722200950177	8249001887	1	2	0.717	0.3585

Note: This is an excerpt. Field names were changed/dropped

2.17 Parameters for ITEM_INSPECTION

AC_PARAMETERTAG	Defaults	Description
ACV_AddConstant	1000	Constant used in the denominator of the calculation of the sales rates
AVAIL_STORES_PCT	80	Minimum percentage stores to be present/acceptable on the rawdata
CorrectRelseExtraProm	1.5	Extra tolerance for promotions
CorrectRelseTol_Price	1.5	z-value for the test on price
CorrectRelseTol_Val	1.5	z-value for the test on value
EXCL_MODULES_LIST		Exclude modules list
FilterCV_Price	0.15	Parameter used for shrinking coefficient of variation to prior (for checks on prices)
FilterCV_Vol	0.35	Parameter used for shrinking coefficient of variation to prior (for checks on volumes)
FilterTol_Price	2.5	Parameter used for calculating critical ratio to median (for checks on prices)
FilterTol_Vol	2.5	Parameter used for calculating critical ratio to median (for checks on volumes)
FullerCorrectLimit	0.9	Probability level for the F-test in the Fuller outlier correction procedure
FullerDetectLimit	0.995	Probability level for the F-test in the Fuller outlier detection procedure
FullerMaxTail	50	Maximum size to be considered as a tail
Verbosity	0	Verbose output or not
ibd_char1	ac_area	First characteristic used for defining IBDs
ibd_char2	ac_shoptype	Second characteristic used for defining IBDs
ibd_char3	ac_countryid	Third characteristic used for defining IBDs
promo_dtgroups		List of datatypegroups to be considered as promotional
upd_inquiry	0	Boolean indicating whether the Inquiry table should be updated or not

2.18 Item_Inspection and census

- In census stores we:
 - Don't want to correct atypical observations

- In the case of sample we want to avoid projecting atypical observations in case we think the atypical observation does not represent ZF stores in the population.
- Want to correct mistakes in the data
 - Even in the case of census we want to correct figures that are clearly completely wrong
- **What happens if we do nothing?**
 - Census stores have $X_F=1$ and thus are unlikely to pass the “significance editing”
 - Census stores also means the N increases and that thus they are more likely to become significant
- How can we judge the total effect ?
 - Remember that we have ‘significance’ if: $x_f \times (\text{original_val} - \text{corr_val}) / \text{proj_total_val_IBD} \geq z_value_val \times \text{relse_val_lbd} \times \text{Extra_Tol}$
 - Let’s simplify the notation:
 - $X_f \Rightarrow n/N$
 - $z_value_val \Rightarrow z$
 - $\text{relse_val_lbd} \Rightarrow \text{cv}/\sqrt{n}$
 - $(\text{original_val} - \text{corr_val}) \Rightarrow \text{correction} \Rightarrow c$
 - $\text{proj_total_val_IBD} \Rightarrow \text{Winsorized total} \Rightarrow w$

2.19 Conclusions

- **Developments are ongoing:**
 - Optimization of SQL
 - Use of the Inquiry table
 - Module_Inspection
 - Processing the Rawdata_corrections
- **In principle fully automatic**
- **Comparison corrections with legacy system (BE) difficult but relatively positive**

3 II LDI validation and correction add-on - prices

3.1 II LDI Validation and Correction Add-On – Prices

3.1.1 Project Summary / Scope of the Project

Item inspection in SirVal bypasses UPCs that have five stores or fewer selling in a given week and IBD. To ensure data quality, we need an additional step to handle such cases.

3.1.1.1 Proposal

The proposal relies on a cross-sectional type of validation given the nature of price behavior. This is an adapted cross-sectional analysis designed to ensure faster processing and prevent delays in process completion.

3.1.2 Preprocess

It is advisable to run the bounds after all validation steps are performed and save the result in a reference table.

The process for determining inspection limits runs on the last available period, based on the T period (T+1 being the period for which these limits will be used for inspection). We initiate the process by selecting one IBD / category / store. Once the process is complete, we proceed to the next category until all categories have been processed and inspection limits are calculated.

3.1.2.1 Data Selection

The following data must be present for both the period under inspection and the previous period:

- PERIODID
- IBDID
- STOREID / STORECODE
- CATEGORYID / PC
- MANUFACTURER / MANU
- BRAND / BRN
- PACK WEIGHT / WT1A
- MEASUREMENT UNIT / UNU
- ITEMCODE / P_CODE / NAN_KEY
- NC_PRICE (for period T must be the final price used for production/BAU reporting)

3.1.2.2 Calculations

Two prices are calculated and validated:

Price:

$$\text{price}_{ij} = \text{NC_SLOT2}_{ij} / (\text{NC_SLOT1}_{ij} \times \text{NC_CONV}_{ij})$$

Normalized Price:

$$\text{norm_price}_{ij} = \text{NC_SLOT2}_{ij} / (\text{NC_SLOT1}_{ij} \times \text{NC_CONV}_{ij} \times \text{WT1a}_i)$$

Where:

- i = sign for the item (P_code)
- j = sign for the store
- NC_SLOT2: sales value
- NC_SLOT1: sales units
- NC_CONV: unit conversion factor
- WT1a: pack size or weight

3.1.2.3 Calculating Limits and Distribution

The limits and distribution are calculated iteratively on several levels:

1. CATEGORY, UNU, WT1A, MANU, BRAND, IBDCODE
2. CATEGORY, UNU, WT1A, MANU, BRAND
3. CATEGORY, UNU, WT1A, MANU
4. CATEGORY, UNU, WT1A
5. CATEGORY, UNU

Note: CATEGORY and UNU must be included at all levels.

For each level:

- Filter v_prices (prices > min_price, default = 0)
- Compute:
 - $t_perc_ll = q_low \text{ quantile } (v_prices)$
 - $t_perc_ul = q_high \text{ quantile } (v_prices)$
 - $ND = \text{Numeric Distribution} = \text{count}(\text{prices} > \text{min_price})$

Where:

$$q_low = q_per, q_high = 1 - q_per, \text{ with default } q_per = 0.05$$

If WT1A is not available, v_prices will be norm_prices, but filtering still applies to the raw price.

3.1.3 Validation and Correction Process

- Calculate `NDIBD_i` = number of stores with sales > 0 for the item code in the IBD.
- Items are selected for validation if `NDIBD_i ≤ ldi_tresh` (default = 5).
- For each flagged observation (LDI), assign upper and lower limits from the preprocessing step.
- Use the first level with ≥ 30 valid observations.
- If using normalized prices, the limit is adjusted by multiplying it by the observation's WT1A.

3.1.3.1 Outlier Identification

A price is considered an outlier if:

- `price > upper_limit → high_outlier`
- `price < lower_limit → low_outlier`

3.1.3.2 Correction Logic

- If `low_outlier → corrected_price = lower_limit`
- If `high_outlier → corrected_price = upper_limit`

3.1.3.3 Parameters for Correction Rules

Parameter	Default	Description
<code>adj_direction</code>	3	Direction filter: 1 = ignore low, 2 = ignore high, 3 = correct both
<code>min_init_price</code>	0	Minimum price to apply correction
<code>corr_abs_diff</code>	0	Minimum absolute difference to apply correction
<code>min_ratio</code>	1	Minimum relative difference to apply correction

3.1.4 Output Data

Each correction will generate the following fields:

- PERIODID
- STOREID / STORECODE
- ITEMCODE / P_CODE / NAN_KEY

- PRICE
- NEW PRICE
- RATIO
- Numeric Distribution
- Applied Bound
- Outlier flag

4 II LDI validation and correction add-on - sales

4.1 Problem/ Opportunity

Introduce into Item Inspection a new algorithm to detect outliers also in case of IBDs with less than a minimum number of observations. Currently in case the number Shop items within an IBD is less than 5 the identified corrections are not applied.

4.2 Project Scope

Scope is to identify and correct store items with low distribution.

4.3 Proposal

The process runs on two stages - pre-process and validation and correction during II.

4.3.1 Pre-process

Pre-process operates on data of current and 2 previous period data to generate bounds & cutoffs to be used into current period T. It means pre-process, to generate data usable into current period T, has to be executed on data of the previous 2 periods (T-1 & T-2) using consolidated data. Process has to be executed any time before running the item inspection for the period.

4.3.2 Validation and correction during II

After the core II process is done a check for LDI items is performed and for all items flagged as LDI distances are recalculated. Based on them further decisions are made.

4.4 Sales calculations

4.4.1 Calculating differences for bounds

$$D_{ij} = \frac{Sales_{ij}^{t-1} - Sales_{ij}^{t-2}}{\sqrt{\frac{(0.1 + Sales_{ij}^{t-1} + Sales_{ij}^{t-2})}{2}}}$$

Where:

$Sales_{ij}$ = sales units (Converted Units (NC_SLOT1*NC_CONV))

i = sign for the item (f_nan_key)

j = sign for the store

T-1 = previous period

T-2 = period before previous period

4.4.2 Calculating bounds

The bounds are calculated during the pre-process stage only. In order to calculate limits and bounds we need to:

- Calculate statistics – once on *CAT/IBD (bounds)* and once on *CAT only level (catbounds)* - using the D_{ij} calculated above and calculate the first (Q_1), the Median (MED or Q_2) and the third quartile (Q_3) of the distribution of D_{ij} (only non-zero values of d_{ij} participate) and IQR($Q_3 - Q_1$) and handling(count).
- Calculate bounds – must calculate cat/ibd bound and catbound independently using the following formula:

$$bound = \max(\text{minimum_bound}, (\text{bound_coefficient} * IQR + MED))$$

- If the handling in cat/ibd group is less than 5, then instead of bounds – catbounds are used.
- Calculate Numeric Distribution on IBD/f_nan_key level – ND_{IBD_i} , number of stores with sales > 0, for the itemcode in IBD

For the current period after running ii a check for the LDI items is performed. For items which distribution in ibd is less than *ldi_tresh* differences are recalculated and those differences are then compared to the calculated bounds during the pre-process.

4.4.3 Calculating differences for bounds LDI items current period

$$D_{ij} = \frac{Sales_{ij}^t - Sales_{ij}^{t-1}}{\sqrt{\frac{(0.1 + Sales_{ij}^t + Sales_{ij}^{t-1})}{2}}}$$

Where Sales are the Converted Units (NC_SLOT1*NC_CONV)

T = current period

T-1 = previous period

ij is the item I within Store J

4.4.4 Outlier detection

A store / item has to be considered outlier if the absolute value of the difference is greater than the absolute value of the bound calculated by the preprocess.

$$abs(D_{ij}) > abs(bound) \wedge ND_{IBD_i} \leq ldi - tresh \implies outlier$$

4.4.5 Corrections

Based on the above calculations we apply correction to all identified outliers following the below formula:

$$CorrSales = ABS\left(\frac{-\left(4 * Sales_{ij}^{t-1} + (-|bound|)^2\right) - SIGN(D_{ij}) * \sqrt{\left(4 * Sales_{ij}^{t-1} + (-|bound|)^2\right)^2 - 4 * 2 * (2 * Sales_{ij}^{t-1} - (-|bound|)^2 * 0.1 - (-|bound|)^2 * Sales_{ij}^{t-1})}}{4}\right)$$

4.5 New Items cross sectional validation

As the algorithm calculates difference (Dij) and further on applies correction on storecode-itemcode level, it is highly likely new item (for the store) to be smoothed even if the sold units are acceptable. This validation is active and could be used only for countries which run and maintain item activity process.

To detect if the new item within the store will be treated as an outlier cross sectional validation on the volume is used. The groups for analysis and comparison are built on two levels:

- IBD-CAT-MAN-BRN
- IBD-CAT (if on the first level we have less than certain number of observations (newitem_min_obs= 30))

Volume = Sales * WT1A

Where:

Sales = sales units (Converted Units (NC_SLOT1*NC_CONV))

WT1A = weight multiplier, actual weight, quantity etc.

4.5.1 Detection and correction

- Cutoff estimation: For each group 95 percentile of the volume is calculated. If IBD-CAT-MAN-BRN has less than certain number of observations (newitem_min_obs= 30), the cutoff calculated at IBD-CAT is used instead.

Note: Each observation is an individual volume, for all sales volumes higher than 0 in ascending order. When the desired quantile lies between two data points i and j , the method of interpolation used is linear: $i + (j - i) * \text{fraction}$, where fraction is the fractional part of the index

- Correction: Three conditions are checked in order observation to be corrected:

- Observation is confirmed as new (flag > 0)
- Its volume is higher than the cutoff
- It has been identified as outlier in the main II LDI algorithm

If all those conditions are met, the observation is corrected to have volume = cutoff and its sales are recalculated to follow:

$$\text{ni_corr_sales} = \text{cutoff} / \text{WT1A} / \text{NC_CONV}$$

If the observation is confirmed as new, was initially proposed for correction by II LDI algorithm, but its volume is lower than cutoff, then its sales are reversed to their initial value (Sales)

Additional control on corrections

Couple of parameters are introduced to further control whether the corrections will be applied or not:

- **adj_direction** - It is a new Parameter used to drive the final selection of the corrections to perform.

Correction Size = Initial Sales Units – New Sales Units

adj_direction	Check	Action
1	Correction size < 0	Ignore correction
2	Correction size > 0	Ignore correction
3		Proceed using all new calculated units

Values of the new parameter are :

1 : Apply only corrections where new Units are greater than the original one

2 : Apply only corrections where new Units are less than the original one

3 : Apply all detected corrections

- **min_init_sales** - if the initial sales are less than or equal to that parameter no corrections will be applied
- **corr_abs_diff**

$$\text{corr_abs_diff} = \text{abs}(\text{corr_sales} - \text{sales})$$

- corr_rel_diff

$$\text{corr_rel_diff} = (\text{sales} / \text{corr_sales}) - 1$$

4.6 Prices calculation

in development

4.7 Parameters

AC_PARAMETERTAG	Defaults	Description
Idi_tresh	5	Max Numeric distribution to define LDI
minimum_bound	4.5	Minimum Bound Value to assign
Idi_bound_coeff	1.9992	Coefficient used to calculate the Bound
Idi_minibd_hndl	5	minimum number of handlers by IBD/Module to use the bounds on IBD level
min_init_sales	50	Minimum initial sales
corr_abs_diff	50	Absolute Difference
corr_rel_diff	20	Relative Difference
adj_direction	3	To drive actions in case of positive or negative difference
newitem_min_obs	30	minimum number of handlers by IBD/Module/Manufacturer/Brand to use the cutoffs on IBD/Module level

- Idi_tresh- max numerical distribution to identify low distributed items. It is calculated on the Total Country / Item Inspection filter session run.

- `minimum_bound`- It is a parameter to define the minimum Bound Value to assign. If the calculated bound is lower than that parameter this value will be used as bound to identify if the sales distance is outlier or not.
- `ldi_bound_coeff` - default of 1.9992. This coefficient controls the sensitivity of the range and hence the decision rule. A bigger scale would make the outliers to be considered as data points while a smaller one would make some of the data points to be perceived as outliers. Also the calculated bound at the end affects the correction size.
- `ldi_minibd_hndl` - two sets of bounds are calculated - one on item/IBD (bound) and one on cat/IBD (cat bound). If the handlers on an item are less than this parameter the catbound will be used instead.
- `min_init_sales` - if the initial sales are less than that parameter any correction will be ignored
- `corr_abs_diff` - if the correction size is less than that parameter the correction will be ignored
- `corr_rel_diff` - if the relative difference between initial sales and corrected sales is less than that parameter the correction will be ignored

4.8 Links

- [AppDev DM document](#)²

2. https://docs.google.com/document/d/1yy5_8ZNVZNB_UuIV25urmhg9XBRh2TNqxcdt09hWFI/edit

5 II SOP

5.1 Business Need

The purpose of this page is to provide the Standard Operating Procedure for processing Item Inspection:

- Item Inspection
- Raw data correction (because of Item Inspection)

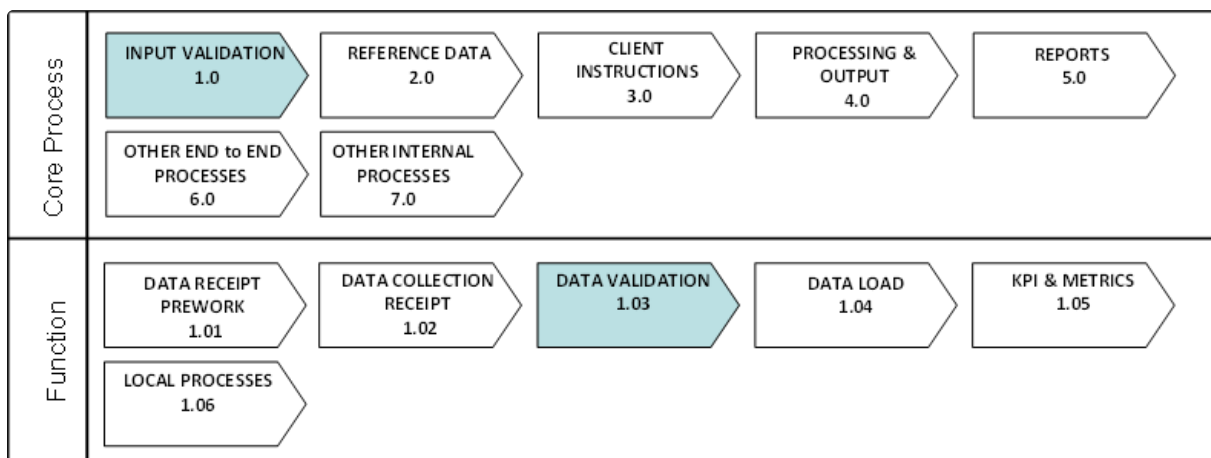
Incoming retailer data can contain extreme sales/prices at the item level.

In a sample environment, these outliers and errors must be corrected to avoid mistakes in the extrapolated data. In a census environment, only obvious errors in volumes/prices are corrected.

The process of detecting such outliers in price and volume is Item Inspection.

The corrections proposed by the Item Inspection process will be applied through the Raw Data correction process.

5.2 High-Level Process Flow



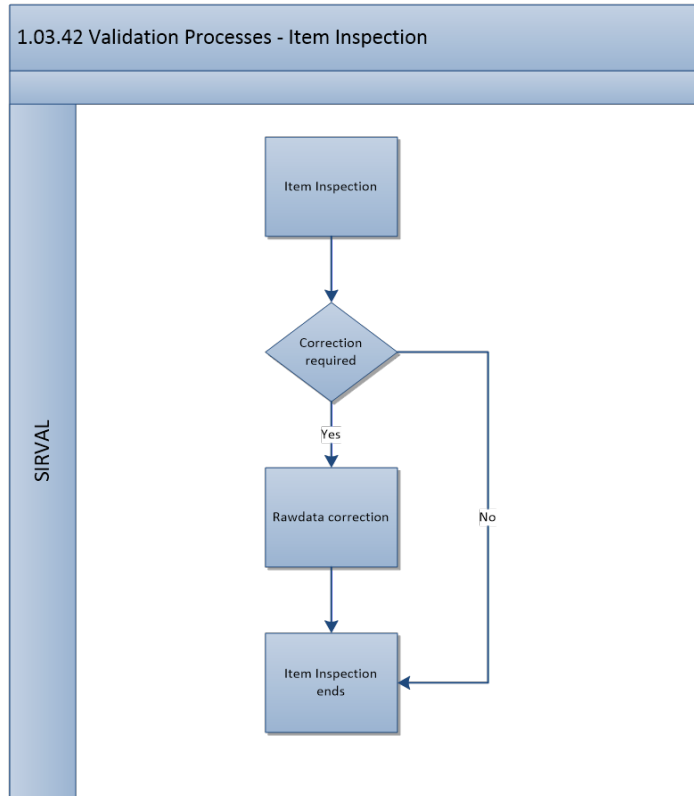
5.3 SOP Purpose

The purpose of this document is to describe the standard procedure of processing Item inspection:

- Item Inspection
- Raw data correction (as result of Item Inspection)

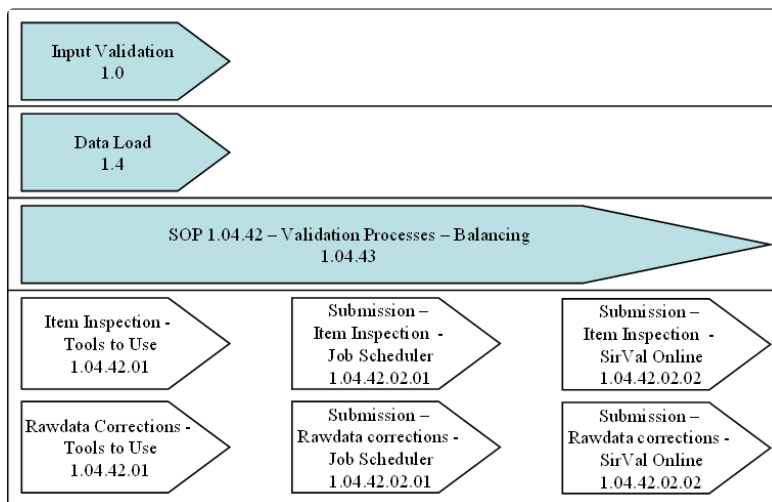
5.4 SOP Process Flow Chart

Process flow that describes the tasks to be performed and the “Roles” who are accountable for completing the tasks.



5.5 Detailed Steps

See the detailed steps regarding the SOP:



- Step 1.03.42.01.01 – Item Inspection – Tools to use.
 - Two different tools are used for submitting Item Inspection:
 - i. Job Scheduler: Used on regular schedule, most of the time this job is included in the pre-production process. Operator need not process this separately or have any reminders setup.
 2. SIRVAL Web Interface: Used on an ad hoc basis for urgent requirements only
- Step 1.03.42.01.02 – Item Inspection – Rules and Restrictions.
 - Rules and restrictions:
 - i. Decision to run the process as part of the pre-production processes for a given week is taken based on the following rules:
 - For all sample stores, it is a must to submit Item Inspection. Since the numbers from Sample services represent the whole universe, the undetected outliers, if not corrected, will be extrapolated in the final output.
 - The rule coming from MSCI is that for census shops the outliers should be left as they are. Only obvious errors in the incoming retailer data are subject of corrections by Item Inspection
 - Parameters must be reviewed on a regular basis, at least twice a year
 1. Item Inspection is a lengthy process. If the processing times are too long, a GO QLS should be opened to discuss the most optimal filter setting.

2. Item Inspection should only run once on the same data set in correction mode for a particular period. A rerun would generate a different distribution of sales/ prices, and the risk exists to make unnecessary corrections.
3. In any BAU situation, template jobs should be used in the Job Scheduler, including automatic submission of the Raw data Correction

- Step 1.03.42.01.03 – Item Inspection – Job Scheduler

- Log in to the SIRVAL Web Interface using your credentials. Select the appropriate country and period set.

In BAU, this process must be submitted, including “Automatic Corrections.”

To submit through the Job Scheduler:

- Go to Job Scheduling a Job Library, here the operator can find all the template jobs which are already created (In case a template job is not available, and the operator needs to create it, then kindly refer to the ‘Functional document Job Scheduler’)a
- The Item Inspection process is part of the “Pre Production Job” for the respective country channel
- According to the operator’s requirement and the Job specification, the correct template should be selected
- At the time of submission, the operator needs to select the appropriate period from the “Change period” option, Start time, End time, and Job class & priority
- The submitted job can be monitored from Job Scheduling a Job Monitor

Parameters for every process included in the Template Job have been set in advance. The operator shouldn’t change these parameters. The order of the processes and the parameters of a Template Job are fixed to avoid errors in BAU.

The benefit of using Job Scheduler templates is that:

- i. Manual and repeated tasks can be automated.
- ii. Interdependent processes are clubbed together in one job.

The process automatically stops in case there is an error in any of the clubbed processes, and the subsequent process wouldn’t start.

- Step 1.03.42.01.04 – Item Inspection – SIRVAL Web Interface

- Log in to the SIRVAL Web Interface with your respective details. Select the appropriate country and period set.
 - Go to “Process submission” menu → “Item Level Validation” → “Item Inspection”
 - The operator needs to select all the appropriate filters

- The option to submit the processes of Raw data Corrections is present in the Item Inspection interface.

The operator must monitor the process and take appropriate actions in the event of an abend – refer to Common Error & Resolutions.

The benefit of using SIRVAL Web Interface is that –

1. It allows executing one process at a time
2. It allows flexibility to change the filters as per the requirement and thus it's used for processing ad hoc or urgent requests

- Step 1.03.42.02.01 – Raw data Correction – Tools to use

- Raw data Corrections can be submitted using the following tools:
 - Item Inspection Interface
 - Raw data Correction Interface

- Step 1.03.42.02.02 – Raw data Correction – Item Inspection Interface

- Log in to the SIRVAL Web Interface using your credentials. Select the appropriate country and period set.
 - Option to submit the processes of Raw data Corrections is present in the same Item Inspection interface, “Automatically Execute the corrections”. It executes Raw data Correction automatically after the Item Inspection process ends

The operator must monitor the process and take appropriate actions in the event of an abend—refer to Common Error & Resolutions.

The benefit of submission from the Item Inspection interface is that –

- The operator need not look after the raw data correction process as it submits the corrections automatically

- Step 1.03.42.02.03 – Raw data Correction – Raw data Correction Interface

- This option should only be used in following scenarios:
 - New country setup
 - New channel setup
 - Any adhoc cases e.g. correction needs to be checked before submission.

Login to the SIRVAL Web Interface with respective details. Select the appropriate country and period set.

- Go to "Process submission" menu → "Raw data correction"
- The Raw data correction interface will be displayed with information related to the suggested corrections as there are Ticket number, Process, Period, Description etc....
- The operator has to select the appropriate ticket (as available in Sysactivity log of Item Inspection)

Operator must monitor the process and take the appropriate actions in case of abends – Refer to Common Error & Resolutions

Advantages of using Raw data Correction interface:

- i. The operator may check the corrections and decide if all of them need to be corrected or leave the data as it is.
 - ii. Used for ad hoc request.
- Common errors/resolution.
 - In case there is an Abend in any of the processing steps, check the reason for the Abend in the "Sysactivity Log". All of them need to be solved before resubmitting the process.

In addition to the Abend messages, the operator must take care as well of the Warning messages present in the log of Item Inspection.

These warnings, handling about stores ignored in Item Inspection, are mandatory to be solved.

Type of Errors –

- **Input File Error**: Errors in any of the input files used.

Resolution: Contact the applicant of the surgery to receive a corrected file.

- **Process Errors/Issues**: Errors/Issues in the sequence of process submission.

Resolution: Solve the process constraint as explained in the related common error file

- **System Errors/Issues**: Errors/Issues caused by the system.

Resolution: Raise a CCM ticket for Oracle errors.

- **Instruction/Parameter Errors**: Errors that occur due to incorrect parameters or instructions

Resolution: Follow the instructions as explained in the following screen shoots (Taken from the error file)

-

Abend messages				Resolution steps		
Category	PROCESSID	EXCEPTIONID	Error message	Action 1	Action 2	Resolution
Instruction/parameter setting	ITEM_INSPECTION		20003 Nr of acceptable stores too low compared to nr of received stores for current period	Contact the country and decide on the actions to take		
Instruction/parameter setting	ITEM_INSPECTION		20004 NULL value in parameters	Missing parameter - update parametervaluestable		
Instruction/parameter setting	ITEM_INSPECTION		20005 Nr of received stores in current period is too low compared to previous period	Contact the country and decide on the actions to take		
Instruction/parameter setting	ITEM_INSPECTION		20006 The parameter set is NULL unable to proceed!	Missing parameter - update parametervaluestable		
Warning messages						
Category	PROCESSID	EXCEPTIONID	Warning message	Action 1	Action 2	Resolution
Instruction/parameter setting	ITEM_INSPECTION		20703 Store Skipped: Invalid cell: 11000200069	Send across an email to Start Ops asking to provide the correct cell information		
Instruction/parameter setting	ITEM_INSPECTION		20703 Store Skipped: Invalid acv: 1000033914	Send across an email to Start Ops asking to provide the correct ACV information		
Instruction/parameter setting	ITEM_INSPECTION		20704 Store skipped (Because of missing store info) although store data was found: 11000200069	Send an email to the IV department to ask for all relevant store information		

A	B	C	D	E	F	G
Category	PROCESSID	EXCEPTIONID	Error message	Action 1	Action 2	Resolution
Input File Error	RAWDATA_CONNECTION	20001	Unknown operation in the readdata_corrections table	Check the file with appropriate tool	Correct the application of the surgery for the required action - correct the file with appropriate operation symbol and record	
Input File Error	RAWDATA_CONNECTION	20004	No cell was given for an operation which requires a cell	Check the store with FOUNDATION or Not processed and set the store to ok or salvage it	Contact the applicant of the surgery for the required action	
Process Error/Issue	RAWDATA_CONNECTION	20008	Returning status for this Store: Pending Output is NOT GOOD.	Check the store with FOUNDATION or Not processed and set the store to ok or salvage it	Contact the applicant of the surgery for the required action	
Process Error/Issue	RAWDATA_CONNECTION	20009	Too or more stores are not null in a given cell operation	For these operation which values are not required	Remove the store value in the file and set it and select again the corrections	
Process Error/Issue	RAWDATA_CONNECTION	20010	Unknown operation on Readdata failed	Record does not exist which we are going to delete	Contact the applicant of the surgery for the required action	
Input File Error	RAWDATA_CONNECTION	20011	Unprocessable LC operation with both input and output	Correct/Remove the record with input & output	Contact the applicant of the surgery for the required action	
Input File Error	RAWDATA_CONNECTION	20012	Unknown operation on Readdata failed	Record does not exist which we are going to delete	Contact the applicant of the surgery for the required action	
Input File Error	RAWDATA_CONNECTION	20013	If the OPERATION is a CA the AC_CODEGROUP must be NULL	Remove AC_CODEGROUP from file		
Input File Error	RAWDATA_CONNECTION	20014	If the OPERATION is a CA the AC_CODEGROUP must be NULL	Remove AC_CODEGROUP from file		
Input File Error	RAWDATA_CONNECTION	20015	If the OPERATION is a CA the AC_CODEGROUP must be NULL	Remove AC_CODEGROUP from file		
Input File Error	RAWDATA_CONNECTION	20016	There is an inconsistency between the stores and the service selected	Check whether appropriate service is selected or not for the surgery if not select correct service	Contact the applicant of the surgery for the required action	resubmit the correction
Process Error/Issue	RAWDATA_CONNECTION	20019	Period not open in service table	Remove the correction for the periods which are too old or open the period if it is required	Contact the applicant of the surgery in case of corrections for a too old period	
Process Error/Issue	RAWDATA_CONNECTION	20020	Ship does not exist in Stores table	Submitted corrections for the stores which is not present in the stores table	Contact the applicant of the surgery for the required action	
System Error	RAWDATA_CONNECTION	001	unprocessed ORACLE error: ORA-00001: unique constraint (VLDPROCESSE_DT_CORR_RAWDATACORRECTION_IDT1) violated	Remove CCM		
System Error	RAWDATA_CONNECTION	002	unprocessed ORACLE error: ORA-00001: unique constraint (VLDPROCESSE_DT_CORR_RAWDATACORRECTION_IDT1) violated	Remove CCM		

A GO QLS should be raised in case the root cause/solution of the issue is unclear. Only when the root cause of the issue is clearly technical, a CCM ticket should be opened. If the same type of issues occurs frequently, the operator should open a GO QLS as well (even in case of a technical issue).